

第 2・4 回 モジュール演算

執筆: 安永憲司

1 四則演算の計算量

整数同士の四則演算について考える。各整数は二進法で表し、整数 x, y はそれぞれ n ビットとする。

足し算 $x + y$ の結果は高々 $n + 1$ ビットで表現できる。計算はいわゆる筆算を行えば良く、計算量は $O(n)$ である。

掛け算 xy の結果は高々 $2n$ ビットで表現でき、筆算で計算すると計算量は $O(n^2)$ である。掛け算を再帰アルゴリズムで計算する方法がある。掛け算 $x \cdot y$ は、

$$x \cdot y = \begin{cases} 2(x \cdot \lfloor y/2 \rfloor) & y \text{ が偶数のとき} \\ x + 2(x \cdot \lfloor y/2 \rfloor) & y \text{ が奇数のとき} \end{cases}$$

と書き直すことができる。この式に従うと、以下の再帰アルゴリズムで計算できる。

```

1  int multiply ( int x, int y ) {
2      if ( y == 0 ) {          // y が 0 のとき
3          return 0;           // 0 を返す
4      }
5      z = multiply( x, y/2 );  // z を x * (y/2) とする
6      if ( y % 2 == 0 ) {      // y が偶数のとき
7          return 2 * z;        // 2 * z を返す
8      } else {                 // y が奇数のとき
9          return x + 2 * z;     // x + 2 * z を返す
10     }
11 }
```

このアルゴリズムの計算量を考えよう。毎回の呼び出しで y の値は半分になっており、 y は n ビット整数であるため、合計 n 回呼び出す。各呼び出しにおける計算は、整数を 2 で割ったり、2 を掛けたり、足したりするだけであるため計算量は $O(n)$ であり、合計 $O(n^2)$ の計算量である。(二進法表現の整数に対して 2 を掛けたり割ったりする演算はビットシフトに対応する。) 掛け算の計算量は、Karatsuba 法などによりさらに削減することができる。一方で、足し算や引き算に関しては、 x, y を読み込むだけで $O(n)$ しかかるため、漸近的にはこれ以上削減できない。

割り算は、 $x = yq + r$ かつ $r < y$ を満たす非負整数 q, r を求めることに対応する (q は商、 r は余り)。割り算の筆算を用いれば計算できるが、ここでは 2 の掛け算・割り算・偶奇判定など簡単な演算で実現される計算方法を見てみよう。

$$\text{divide}(x, y) = (q, r) \quad \Leftrightarrow \quad q \text{ と } r \text{ は, } x = yq + r, r < y \text{ を満たす非負整数}$$

と定め、

$$\text{divide}(\lfloor x/2 \rfloor, y) = (q', r') \quad \Leftrightarrow \quad q' \text{ と } r' \text{ は, } \lfloor x/2 \rfloor = yq' + r', r' < y \text{ を満たす非負整数}$$

とおく。このとき、以下のように場合分けできる。

1. x が偶数 ($x = 2z$) のとき, $z = q'y + r'$ であり, $x = 2z = 2q'y + 2r'$ が得られる.
 - (a) $2r' \geq y$ ならば, $x = (2q' + 1)y + 2r' - y$ とすれば $2r' - y < y$ のため, $(q, r) = (2q' + 1, 2r' - y)$
 - (b) $2r' < y$ ならば, $x = 2q'y + 2r'$ かつ $2r' < y$ であるため, $(q, r) = (2q', 2r')$
2. x が奇数 ($x = 2z + 1$) のとき, $z = q'y + r'$ であり, $x = 2q'y + 2r' + 1$ が得られる.
 - (a) $2r' + 1 \geq y$ ならば, $x = (2q' + 1)y + 2r' + 1 - y$ とすれば $2r' + 1 - y < y$ のため, $(q, r) = (2q' + 1, 2r' + 1 - y)$
 - (b) $2r' + 1 < y$ ならば, $x = 2q'y + 2r' + 1$ かつ $2r' + 1 < y$ であるため, $(q, r) = (2q', 2r' + 1)$

上記にもとづくのが以下のアルゴリズムである. このプログラムはこのままでは動作しないことに注意しよう.

```

1  /*****
2  割り算
3  入力: 整数  $x \geq 0, y \geq 1$ 
4  出力:  $x$  を  $y$  で割ったときの商  $q$  と余り  $r$ 
5  *****/
6  void divide ( int x, int y ) {
7      if ( x == 0 ) {
8          return (q, r) = (0, 0);    //  $x$  が 0 なら  $(q, r) = (0, 0)$ 
9      }
10     (q, r) = divide( x/2, y );    //  $x$  を半分にして再帰呼び出しで  $(q, r)$  を求める
11     q = 2 * q;                    // 得られた  $q$  を 2 倍
12     r = 2 * r;                    // 得られた  $r$  を 2 倍
13     if ( x % 2 == 1 ) r = r + 1;  //  $x$  が奇数なら  $r$  に 1 加える
14     if ( r >= y ) {                //  $r$  が  $y$  以上のとき
15         r = r - y;                //  $r$  から  $y$  を引く
16         q = q + 1;                //  $q$  に 1 加える
17     }
18     return (q, r);                //  $(q, r)$  を返す
19 }
```

x の値が再帰呼び出しで毎回半分になっているため, 高々 n 回しか呼び出されない. 各呼び出しで $O(n)$ 計算量の演算を行うため, 計算量の合計は $O(n^2)$ である.

2 モジュロ演算 (剰余演算)

モジュロ演算 (modular arithmetic) は整数 N で割ったときの余りを考える演算である. 整数 x, y が法 N のもとで合同 (congruent) であるとは, $x \bmod N = y \bmod N$ を満たすときであり,

$$x \equiv y \pmod{N}$$

と書く. もし, $x \equiv x' \pmod{N}$ かつ $y \equiv y' \pmod{N}$ であれば,

$$x + y \equiv x' + y' \pmod{N}, \quad xy \equiv x'y' \pmod{N}$$

が成り立つ. 以下の通り, 結合法則, 交換法則, 分配法則も成り立つ.

$$x + (y + z) \equiv (x + y) + z \pmod{N}$$

$$xy \equiv yx \pmod{N}$$

$$x(y + z) \equiv xy + xz \pmod{N}$$

法 N のもとでの整数 x, y の足し算 $x + y \bmod N$ の計算量は、 $n = \lceil \log_2 N \rceil$ に対し、 $O(n)$ である。掛け算 $xy \bmod N$ の計算は、 xy を求めてから剰余を計算すればよく、掛け算を実現する計算量と同程度である。（上記の方法では $O(n^2)$ である。）割り算について、整数上では 0 で割れないことを気にすればよかったが、モジュロ演算ではさらに多くの値で割れない可能性がある。計算方法は後で説明する。

3 モジュロ冪乗演算

冪乗 (exponentiation) $x^y \bmod N$ を求める方法を考える。単純に x^y を求めてから剰余を求めようとすると、 x^y のビット長だけで $\log_2 N^N = N \log_2 N = O(n2^n)$ であり、覚えておくことさえ大変である。数が大きくなりすぎないように、常に剰余の値を覚えておくと、

$$x \bmod N \rightarrow x^2 \bmod N \rightarrow x^3 \bmod N \rightarrow x^4 \bmod N \rightarrow \cdots \rightarrow x^y \bmod N$$

と計算できるが、 $y \leq N = O(2^n)$ 回の掛け算をすることになる。ここで、二乗演算を考えると、

$$x \bmod N \rightarrow x^2 \bmod N \rightarrow x^4 \bmod N \rightarrow x^8 \bmod N \rightarrow \cdots \rightarrow x^{2^{\lceil \log_2 y \rceil}} \bmod N$$

のようにして、 $\lceil \log_2 y \rceil = O(\log_2 N) = O(n)$ 回の掛け算で $x^y \bmod N$ に近い値が求められることがわかる。これを利用したのが以下の**繰り返し二乗法** (repeated squaring) である。これは、

$$x^y = \begin{cases} (x^{\lfloor y/2 \rfloor})^2 & y \text{ が偶数のとき} \\ x \cdot (x^{\lfloor y/2 \rfloor})^2 & y \text{ が奇数のとき} \end{cases}$$

という関係を再帰的に用いて計算している。

```

1  /*****
2  繰り返し二乗法（再帰版）
3  入力：整数  $x, y \geq 0, N > 1$ 
4  出力： $x^y \bmod N$ 
5  *****/
6  int modexp1 ( int x, int y, int N ) {
7      int z;
8      if ( y == 0 ) {          //  $y$  が 0 のとき
9          return 1;           // 1 を返す
10     }
11     z = modexp1( x, y / 2, N ); //  $z = x^{(y/2)} \bmod N$  とする
12     if ( (y % 2) == 0 ) {      //  $y$  が偶数のとき
13         return z * z % N;      //  $z^2 \bmod N$  を返す
14     } else {                  //  $y$  が奇数のとき
15         return x * z * z % N;  //  $x(z^2) \bmod N$  を返す
16     }
17 }
```

計算量は、再帰呼び出しが $O(\log_2 N) = O(n)$ 回あり、各呼び出しにおいて掛け算の計算量 $O(n^2)$ がかかるため、全体で $O(n^3)$ である。

例 3.1. $6^{25} \bmod 2025$ を計算するとき、`modexp1(6, 25, 2025)` の呼び出しを `modexp(25)` と表せば、

$$\text{modexp}(25) \rightarrow \text{modexp}(12) \rightarrow \text{modexp}(6) \rightarrow \text{modexp}(3) \rightarrow \text{modexp}(1) \rightarrow \text{modexp}(0)$$

の順に呼び出される．返り値は

$$\begin{aligned}\text{modexp}(0) &= 1 \\ \text{modexp}(1) &= 6 * 1^2 = 6^1 \\ \text{modexp}(3) &= 6 * 6^2 = 6^3 \\ \text{modexp}(6) &= (6^3)^2 = 6^6 \\ \text{modexp}(12) &= (6^6)^2 = 6^{12} \\ \text{modexp}(25) &= 6 * (6^{12})^2 = 6^{25}\end{aligned}$$

と計算される．これは,

$$6^{25} \equiv 6 \cdot (6^{12})^2 \equiv 6 \cdot ((6^6)^2)^2 \equiv 6 \cdot (((6^3)^2)^2)^2 \equiv 6 \cdot (((6 \cdot (6^1)^2)^2)^2)^2 \pmod{2025}$$

という関係を使ってカッコの内側から計算することに対応する．最後の式を,

$$6^1 \cdot (6^0 \cdot (6^0 \cdot (6^1 \cdot (6^1 \cdot 1^2)^2)^2)^2)^2 \pmod{2025}$$

と表すと, 25 の二進法表現である 11001_2 が, 下位ビットから順に 6 の指数に登場することがわかる．最初に 25 が奇数であるため 6^1 が登場し, 次の 12 は偶数であるため 6^0 ．次の 6 も偶数であり 6^0 , 次の 3 は奇数であり 6^1 , 最後の 1 は奇数のため 6^1 が登場する．結果として, $6^1 \cdot (6^0)^2 \cdot (6^0)^4 \cdot (6^1)^8 \cdot (6^1)^{16} = 6^{25}$ が計算される．

再帰関数としてではなく, 単なるループとして計算することもできる．

```

1  /*****
2  繰り返し二乗法 (非再帰版)
3  入力: 整数 x, y >= 0, N > 1
4  出力: x^y mod N
5  *****/
6  int modexp2 ( int x, int y, int N ) {
7      int a = 1, b = x, c = y;           // (a,b,c) = (1,x,y) とする
8      while ( c > 0 ) {                   // c が正である限り以下を繰り返す
9          if ( (c % 2) == 1 ) a = a * b % N; // c が奇数なら a に b を掛ける
10         c = c / 2;                       // c を半分に
11         b = b * b % N;                   // b を二乗する
12     }
13     return a;                           // a を返す
14 }
```

例 3.2. $6^{25} \bmod 2025$ は上記プログラムでは, 以下のように計算される．

a	b	c
1	6^1	25
$1 \times 6^1 = 6^1$	$(6^1)^2 = 6^2$	12
6^1	$(6^2)^2 = 6^4$	6
6^1	$(6^4)^2 = 6^8$	3
$6^1 \times 6^8 = 6^9$	$(6^8)^2 = 6^{16}$	1
$6^9 \times 6^{16} = 6^{25}$	$(6^{16})^2 = 6^{32}$	0

計算としては、以下のように展開した式を左から順に計算している。

$$\begin{aligned} 6^{25} &\equiv 1 \times 6^1 \times 6^8 \times 6^{16} \pmod{2025} \\ &\equiv (6^1)^1 \times (6^2)^0 \times (6^4)^0 \times (6^8)^1 \times (6^{16})^1 \pmod{2025} \end{aligned}$$

モジュロ演算クイズ

クイズ 3.1. 法 13 において、 $5x \equiv 1 \pmod{13}$ を満たす x はどれか。

1. $x = 2$ 2. $x = 5$ 3. $x = 8$ 4. $x = 10$

クイズ 3.2. $2^{20} \bmod 15$ の値として正しいものを選び。

1. 1 2. 4 3. 9 4. 11

クイズ 3.3. 次のうち、 $a \equiv b \pmod{N}$ から必ず成り立つものをすべて選べ。

1. $a^2 \equiv b^2 \pmod{N}$ 3. $a + N \equiv b \pmod{N}$
2. $2a \equiv 2b \pmod{N}$ 4. $a/N \equiv b/N \pmod{N}$

クイズ 3.4. 法 15 において、 $6x \equiv 6y \pmod{15}$ が成り立つとする。このとき、必ず $x \equiv y \pmod{15}$ といえるか。

1. 必ずいえる 2. 必ずしもいえない

クイズ 3.5. $y = 2023$ とする。繰り返し二乗法で $x^y \bmod N$ を計算するとき、命令 $a = a \times b \bmod N$ が実行される回数は、 y の二進法表現の何に等しいか。

1. y のビット長 3. y の二進法表現における 0 の個数
2. y の二進法表現における 1 の個数 4. y の値そのもの

4 ループ不変式

繰り返し二乗法について、特に非再帰版について、上記コードが正しく動作することを確認できるだろうか。再帰関数や while, for などのループ（繰り返し計算）が登場するとき、**ループ不変式** (loop invariant) を使って正しさを確認する方法がある。各ループの始めと終わりに満たすべき条件式のことで、上記に対しては、

$$a \cdot b^c \equiv x^y \pmod{N} \text{ かつ } 0 \leq a < N \text{ かつ } c \geq 0$$

とおく。この条件式に対し、以下を示すことでアルゴリズムの正しさを確認する。

1. **初期化**：最初のループに入る前に正しい。
2. **保持性**：ループ前に正しければ、次のループの直前においても正しい。
3. **事後条件**：ループが終了したとき、ループ不変式はアルゴリズムの正しさに関する事後条件を満たす。

4. **停止性**：ある減少関数が存在し、その値は各ループ後に小さくなり、最終的にループは停止する。
 実際に、繰り返し二乗法（非再帰版）に対して証明してみよう。

1. 初期化：ループの前に $a = 1, b = x, c = y$ と初期化されている。このとき、

$$a \cdot b^c = 1 \cdot x^y = x^y$$

である。また、 $N > 1$ より $0 \leq a < N$ であり、 $y \geq 0$ より $c \geq 0$ である。

2. 保持性：ループにより a, b, c がそれぞれ a', b', c' に変化したとする。ここで示したいのは、

$$a \cdot b^c \equiv x^y \pmod{N}, \quad 0 \leq a < N, \quad c \geq 0$$

であれば、

$$a' \cdot b'^{c'} \equiv x^y \pmod{N}, \quad 0 \leq a' < N, \quad c' \geq 0$$

が成り立つことである。まず、 c が偶数のとき、

$$a' = a, \quad b' = b^2 \bmod N, \quad c' = \lfloor c/2 \rfloor$$

であるため、

$$a' \cdot b'^{c'} \equiv a \cdot (b^2)^{\lfloor c/2 \rfloor} = a \cdot b^c \equiv x^y \pmod{N}$$

である。また、 $0 \leq a' < N$ と $c' \geq 0$ も成り立つ。次に、 c が奇数のとき、

$$a' = ab \bmod N, \quad b' = b^2 \bmod N, \quad c' = \lfloor c/2 \rfloor$$

であるため、

$$a' \cdot b'^{c'} \equiv ab \cdot (b^2)^{\lfloor c/2 \rfloor} = a \cdot b^c \equiv x^y \pmod{N}$$

である。また、 $0 \leq a' < N$ と $c' \geq 0$ も成り立つ。

3. 事後条件：ループが終了するのは $c \leq 0$ のときである。一方、ループ不変式より $c \geq 0$ なので、終了時には $c = 0$ である。このとき、

$$a \equiv a \cdot b^0 \equiv x^y \pmod{N}$$

であり、さらに $0 \leq a < N$ であるため、 $a = x^y \bmod N$ である。したがって、 a を返すことは正しい。

4. 停止性：各ループで c は $\lfloor c/2 \rfloor$ に更新される。したがって、 $c > 0$ である限り c は真に小さくなり、最終的に $c = 0$ となるため、ループは停止する。

5 ユークリッドの互除法

整数 a と b の最大公約数 $\gcd(a, b)$ を求めるアルゴリズムがユークリッドの互除法 (Euclidean algorithm) である。

$$\text{正の整数 } a, b \text{ に対し, } a \geq b \text{ ならば, } \gcd(a, b) = \gcd(b, a \bmod b)$$

という関係を利用する。

```

1  /*****
2   ユークリッドの互除法
3   *****/
4   int euclid ( int a, int b ) { // a は b 以上
5       if ( b == 0 ) {
6           return a;
7       }
8       else return euclid( b, a % b );
9   }

```

計算量について考える。再帰的に、 $(a, b) \rightarrow (b, a \bmod b)$ と呼び出すが、呼び出す値は毎回減っている。

補題 5.1. 正の整数 a, b に対し、 $a \geq b$ ならば、 $a \bmod b < a/2$ である。

この補題より、2 回呼び出すたびに、値が半分になることがわかる。ビット長で言えば、1 ビット短くなる。入力 (a, b) は $2n$ ビットであるため、 $O(n)$ 回の繰り返しで済む。各呼び出しで計算量 $O(n^2)$ の剰余演算を行うため、全体で $O(n^3)$ の計算量である。

5.1 拡張ユークリッドの互除法

補題 5.2. 正の整数 d が正の整数 a, b をともに割り切り、 $d = ax + by$ を満たす整数 x, y が存在するとき、 $d = \gcd(a, b)$ である。

証明. 最初の条件より、 d は a と b の公約数であるため、 $d \leq \gcd(a, b)$ である。一方、 $\gcd(a, b)$ は、 a と b の公約数であるため、 $ax + by$ を割り切り、 $\gcd(a, b) \leq d$ である。□

$\gcd(a, b) = ax + by$ を満たす整数 x, y を求めるアルゴリズムが拡張ユークリッドの互除法 (extended Euclidean algorithm) である。

```

1  /*****
2   拡張ユークリッドの互除法
3   入力: 整数 a, b (a >= b >= 0)
4   出力: 整数 x, y, d : d = gcd(a, b), ax + by = d
5   *****/
6   int ext_euclid ( int a, int b ) { // a は b 以上
7       if ( b == 0 ) {
8           return (1, 0, a);
9       }
10      (x', y', d) = ext_euclid( b, a % b);
11      return (y', x' - (a/b)*y', d);
12  }

```

補題 5.3. 正の整数 a, b に対し、拡張ユークリッドの互除法は、 $\gcd(a, b) = ax + by$ を満たす整数 x, y を返す。

証明. x', y' を無視すればユークリッドの互除法と同じであり、 $d = \gcd(a, b)$ がわかる。

b についての数学的帰納法で示す。 $b = 0$ は自明である。 $b > 0$ のとき、 $\gcd(b, a \bmod b)$ を呼び出して $\gcd(a, b)$ を計算するが、 $a \bmod b < b$ であるため、帰納法の仮定より、

$$\gcd(b, a \bmod b) = bx' + (a \bmod b)y'$$

を満たす x', y' は正しく計算される. このとき, $a \bmod b = a - \lfloor a/b \rfloor b$ に注意すると,

$$\begin{aligned} \gcd(a, b) &= \gcd(b, a \bmod b) \\ &= bx' + (a \bmod b)y' \\ &= bx' + (a - \lfloor a/b \rfloor b)y' \\ &= ay' + b(x' - \lfloor a/b \rfloor y') \end{aligned}$$

であり, $x = y', y = x' - \lfloor a/b \rfloor y'$ とすれば $d = ax + by$ を満たし, アルゴリズムの出力は正しいことがわかる. $\square$

例 5.4. $a = 432, b = 321$ のとき, 拡張ユークリッドの互除法における関数 `ext_euclid` への呼び出し (a, b) とその返り値 (x, y, d) は以下のように計算される.

a	b	x	y	d
432	321	-26	$9 - \lfloor 432/321 \rfloor \times (-26) = 35$	3
321	111	9	$-8 - \lfloor 321/111 \rfloor \times 9 = -26$	3
111	99	-8	$1 - \lfloor 111/99 \rfloor \times (-8) = 9$	3
99	12	1	$0 - \lfloor 99/12 \rfloor \times 1 = -8$	3
12	3	0	$1 - \lfloor 12/3 \rfloor \times 0 = 1$	3
3	0	1	0	3

5.2 モジュロ割り算

一般に, a による割り算は, a の逆元を掛けることで実現する. 法 N において, x が a の**逆元** (*inverse*) であるとは, $ax \equiv 1 \pmod{N}$ を満たすときである. 法 N において逆元は (合同なものを除くと) 高々一つしか存在しないため, a^{-1} と書く.

定理 5.5. 正の整数 a, N に対し, 法 N において a の逆元が存在するとき, かつそのときに限り, $\gcd(a, N) = 1$ である. 逆元が存在するとき, $O(n^3)$ の計算量で見つけられる.

証明. 任意の整数 x に対し, $\gcd(a, N)$ は $ax \bmod N$ を割り切る. したがって, $\gcd(a, N) > 1$ のとき, $ax \not\equiv 1 \pmod{N}$ である. つまり, $\gcd(a, N) = 1$ でないとき, a の逆元は存在しない. また, $\gcd(a, N) = 1$ のとき, 拡張ユークリッドの互除法より, $ax + Ny = 1$ を満たす整数 x, y が存在し, $ax \equiv 1 \pmod{N}$ であり, a の逆元 x が存在する. $\square$

例 5.6. $N = 18$ のとき, 逆元が存在するのは $1, 5, 7, 11, 13, 17$ である. 実際に, $1^{-1} = 1, 5^{-1} = 11, 7^{-1} = 13, 11^{-1} = 5, 13^{-1} = 7, 17^{-1} = 17$ である.

モジュロ演算クイズその2

クイズ 5.1. 次のうち, 法 N において a の逆元が存在するための必要十分条件として正しいものを選べ.

1. a が N の倍数である
2. a と N が互いに素である
3. N が素数である
4. a が奇数である

クイズ 5.2. 法 13 において, 7×5^{-1} の値はどれか.

1. 2

2. 4

3. 8

4. 10

クイズ 5.3. 次のうち, 法 21 において逆元を持つものをすべて選べ.

1. 4

2. 6

3. 14

4. 15

6 素数性判定

定理 6.1 (フェルマーの小定理). 素数 p と整数 $1 \leq a < p$ に対し,

$$a^{p-1} \equiv 1 \pmod{p}$$

が成り立つ.

この定理より, 1 以上 N 未満の整数 a に対し $a^{N-1} \not\equiv 1 \pmod{N}$ を満たせば, N は素数でないことがわかる. 一方で, 合成数 N が $1 \leq a < N$ に対し必ず $a^{N-1} \not\equiv 1 \pmod{N}$ を満たすとは限らない. しかし, 合成数 N は, 多くの a に対して $a^{N-1} \not\equiv 1 \pmod{N}$ であることがわかる. この性質を利用することで, 以下の素数性判定法が思いつく.

```
1  /*****
2   フェルマー検査
3   *****/
4   int fermat ( int N ) {
5       // N 未満の正の整数 a をランダムに選ぶ
6       if ( /* a^{N-1} mod N = 1 のとき */ ) {
7           return 1; // Yes (N を素数) と判定
8       }
9       else return 0; // No (N を合成数) と判定
10  }
```

実際に, 以下が証明できる.

補題 6.2. 整数 $N \geq 2$ に対し, N と互いに素な整数 w で

$$w^{N-1} \not\equiv 1 \pmod{N}$$

を満たすものが存在するとする. このとき, 1 以上 N 未満の整数 a のうち, 少なくとも半分は

$$a^{N-1} \not\equiv 1 \pmod{N}$$

を満たす.

証明. 検査に通る整数の集合を

$$L = \{b \in \{1, \dots, N-1\} : b^{N-1} \equiv 1 \pmod{N}\}$$

とする. 任意の $b \in L$ に対し, $r_b \equiv wb \pmod{N}$ を満たす 1 以上 N 未満の整数 r_b を取る. このとき,

$$r_b^{N-1} \equiv (wb)^{N-1} \equiv w^{N-1}b^{N-1} \equiv w^{N-1} \not\equiv 1 \pmod{N}$$

であるため、 r_b は検査に通らない。また、 w と N は互いに素なので、写像 $b \mapsto r_b$ は単射である。したがって、検査に通らない整数は少なくとも $|L|$ 個存在する。よって、1 以上 N 未満の整数のうち、少なくとも半分は検査に通らない。□

極めて稀な合成数 N として、 N と互いに素であるすべての $a < N$ で検査に通るようなものが存在する。そのような数はカーマイケル数 (Carmichael number) と呼ばれる。上記補題より、カーマイケル数以外の N に対して、

$$\begin{aligned}\Pr(\text{素数 } N \text{ がフェルマー検査で Yes と判定される}) &= 1 \\ \Pr(\text{合成数 } N \text{ がフェルマー検査で Yes と判定される}) &\leq \frac{1}{2}\end{aligned}$$

が成り立つ。検査を k 回繰り返せば、

$$\Pr(\text{合成数 } N \text{ がフェルマー検査で } k \text{ 回とも Yes と判定される}) \leq \frac{1}{2^k}$$

であり、素数性判定の誤り率を k について指数関数的に小さくできる。

例 6.3. $N = 18, 19, 20, 21, 22, 561$ について、 $a^{N-1} \bmod N$ を求めると以下の通り。561($= 3 \cdot 11 \cdot 17$) は最小のカーマイケル数。

a	$a^{17} \bmod 18$	a	$a^{18} \bmod 19$	a	$a^{19} \bmod 20$	a	$a^{20} \bmod 21$	a	$a^{21} \bmod 22$	a	$a^{560} \bmod 561$
1	1	1	1	1	1	1	1	1	1	1	1
2	14	2	1	2	8	2	4	2	2	2	1
3	9	3	1	3	7	3	9	3	3	3	375
4	16	4	1	4	4	4	16	4	4	4	1
5	11	5	1	5	5	5	4	5	5	5	1
6	0	6	1	6	16	6	15	6	6	6	375
7	13	7	1	7	3	7	7	7	7	7	1
8	8	8	1	8	12	8	1	8	8	8	1
9	9	9	1	9	9	9	18	9	9	9	375
10	10	10	1	10	0	10	16	10	10	10	1
11	5	11	1	11	11	11	16	11	11	11	154
12	0	12	1	12	8	12	18	12	12	12	375
13	7	13	1	13	17	13	1	13	13	13	1
14	2	14	1	14	4	14	7	14	14	14	1
15	9	15	1	15	15	15	15	15	15	15	375
16	4	16	1	16	16	16	4	16	16	16	1
17	17	17	1	17	13	17	16	17	17	17	34
				18	12	18	9	18	18	18	375
				19	19	19	4	19	19	19	1

7 素数生成

RSA 暗号などの暗号技術では、大きな素数をランダムに生成したい場面がある。実際に、素数はたくさん存在することが知られている。 x 以下の素数の数を $\pi(x)$ と表すとき、 $\pi(x) \approx x/(\ln x)$ であることが知られている。

定理 7.1 (素数定理).

$$\lim_{x \rightarrow \infty} \frac{\pi(x)}{x/(\ln x)} = 1.$$

ここでは、簡単に証明できる下界を紹介する。

定理 7.2 (Chebyshev の定理). 整数 $m \geq 2$ に対し,

$$\pi(m) \geq \frac{m}{2 \log m}$$

である。

証明. まず, $m = 2n$ の場合を示す. 整数 B を

$$B := \binom{2n}{n} = \frac{(2n)!}{(n!)^2}$$

と定める. $B \geq 2^n$ であり, B を割り切る素数はすべて $2n$ 以下である.

B を素因数分解したとき, 素数 p に対応する指数 (exponent) を $\nu_p(B)$ と表す (例えば, $12 = 2^2 \cdot 3^1$ より, $\nu_2(12) = 2, \nu_3(12) = 1$). このとき, 任意の正の整数 r と素数 p に対し,

$$\nu_p(r!) = \sum_{k \geq 1} \left\lfloor \frac{r}{p^k} \right\rfloor$$

が成り立つ (証明は省略). これを B に対して適用すると,

$$\nu_p(B) = \sum_{k \geq 1} \left(\left\lfloor \frac{2n}{p^k} \right\rfloor - 2 \left\lfloor \frac{n}{p^k} \right\rfloor \right)$$

が得られる. 各項は 0 または 1 であり, $k > \log(2n)/\log p$ に対しては 0 であるため, $\nu_p(B) \leq \log(2n)/\log p$ である. このとき,

$$\begin{aligned} \pi(2n) \log(2n) &= \sum_{\substack{\text{素数 } p \leq 2n}} \frac{\log(2n)}{\log p} \log p \\ &\geq \sum_{\substack{\text{素数 } p \leq 2n}} \nu_p(B) \log p \\ &= \log B \\ &\geq n \end{aligned}$$

である. これより, $\pi(2n) \geq 2n/2 \log(2n)$ であり, m が偶数の場合に示せた. m が 3 以上の奇数 ($m = 2n - 1, n \geq 2$) のとき, $x/\log x$ が $x \geq 3$ で増加関数であることから,

$$\begin{aligned} \pi(2n - 1) &= \pi(2n) \\ &\geq \frac{2n}{2 \log(2n)} \\ &\geq \frac{(2n - 1)}{2 \log(2n - 1)} \end{aligned}$$

であり, 奇数の場合も示せた. □

素数定理より, n ビットの整数をランダムに選んだとき, それが素数である確率はおおよそ $1/(\ln 2^n) \approx 1.44/n$ である. ランダムに整数 N を選び, N の素数性判定を行い, 検査に通れば N を素数として出力し, 検査に通らなければ新たに整数を選んで検査を行う. この手続きによりランダムに素数を生成することができる. 正しく素数が選ばれる確率は $1/n$ 程度あるため, 期待値として $O(n)$ 回の繰り返しで素数を見つけることができる.

より強力な素数性判定法として, Miller-Rabin 法が知られている. フェルマー検査と同様に確率的な判定法である. 素数性判定については, 決定性のアルゴリズムも知られているが, 効率面から確率的アルゴリズムが用いられている.

素数クイズ

クイズ 7.1. フェルマーの小定理を利用した素数性判定法（フェルマー検査）について正しいものをすべて選べ.

1. 素数 p に対して任意の $1 \leq a < p$ で $a^{p-1} \equiv 1 \pmod{p}$ が成り立つ
2. 合成数 N に対して任意の $1 \leq a < N$ で $a^{N-1} \equiv 1 \pmod{N}$ が成り立つ
3. 合成数 N でも, N と互いに素な任意の $1 \leq a < N$ で $a^{N-1} \equiv 1 \pmod{N}$ となることがある
4. フェルマー検査は必ず正しい素数判定を行う

クイズ 7.2. 素数定理からわかることとして最も適切なものを選べ.

1. n ビット整数をランダムに選ぶと, 素数である確率はおおよそ $1/n$ である
2. n ビット整数をランダムに選ぶと, 素数である確率はおおよそ $1/2^n$ である
3. n ビット整数をランダムに選ぶと, ほとんど必ず素数である
4. n ビット整数をランダムに選ぶと, 素数である確率はおおよそ $1/n^2$ である

8 モジュロ演算の演習問題

演習問題 8.1. 等比級数の和は収束するが, 調和級数 (harmonic series) の和は発散する. つまり,

$$\sum_{i=1}^{\infty} \frac{1}{i} = \infty.$$

n が十分に大きいとき, ある定数 $\gamma = 0.5772 \dots$ を用いて,

$$\sum_{i=1}^n \frac{1}{i} \approx \ln n + \gamma$$

と近似できる. ここで, $\ln$ は自然対数である.

$$\sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$$

を示せ. (ヒント: 上界と下界を示すため, 分母を2のべき乗で揃えることを考えよう.)

演習問題 8.2. $a \equiv b \pmod{N}$ であり M が N を割り切るとき, $a \equiv b \pmod{M}$ を示せ.

演習問題 8.3. $4^{1536} - 9^{4824}$ は 35 で割り切れるか.

演習問題 8.4. $2^{2^{2006}} \bmod 3$ の値を求めよ.

演習問題 8.5. $2^{125} \bmod 127$ の値を求めよ.

演習問題 8.6. 法 79 における 20 の逆元, 法 62 における 3 の逆元, 法 91 における 21 の逆元, 法 23 における 5 の逆元を求めよ.

演習問題 8.7. 整数 n に対し, $n!$ を計算するアルゴリズムを与え, その計算量を評価せよ.

演習問題 8.8. 以下の定理 8.1 を用いて, 素因数分解の一意性を証明せよ. つまり, 任意の合成数 a に対し,

$$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}$$

を満たす素数 $p_1 < \cdots < p_r$ ならびに正の整数 $e_1, \dots, e_r$ が一意に存在する.

定理 8.1. 任意の素数 p と整数 a, b に対し, $p \mid ab$ ならば $p \mid a$ もしくは $p \mid b$ (もしくはその両方) である. ここで, $x \mid y$ は, x が y を割り切ることを表す.

演習問題 8.9. 素数 p と正の整数 $k < p$ に対し, $p \mid \binom{p}{k}$ を示せ. 結果として, 整数 a, b と素数 p に対し, $(a+b)^p \equiv a^p + b^p \pmod{p}$ が成り立つ.

演習問題 8.10. 整数 a, b に対し, $a \mid b$ であるとき, $(x \bmod b) \bmod a = x \bmod a$ を示せ. また, $a \mid b$ であるとき, $x \equiv y \pmod{b}$ ならば $x \equiv y \pmod{a}$ であることを示せ.

演習問題 8.11. 整数 a, b, c に対し, $\gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c)$ を示せ.

クイズの答え

クイズ 3.1. $5 \times 8 = 40 \equiv 1 \pmod{13}$ より, 答えは 3.

クイズ 3.2. $2^4 = 16 \equiv 1 \pmod{15}$ であるため,

$$2^{20} = (2^4)^5 \equiv 1 \pmod{15}.$$

答えは 1.

クイズ 3.3. $a \equiv b \pmod{N}$ なら, 両辺に同じ数を足しても, 掛けても, 合同性は保たれる. したがって, 1, 2, 3 は成り立つ. 一方, 割り算は一般にはできない. 答えは 1, 2, 3.

クイズ 3.4. 必ずしもいえない. 例えば $x = 1, y = 6$ とすると,

$$6x = 6, \quad 6y = 36 \equiv 6 \pmod{15}$$

であるが,

$$1 \not\equiv 6 \pmod{15}$$

である. 答えは 2.

クイズ 3.5. 命令

$$a = a \times b \bmod N$$

は, 現在の c が奇数のときだけ実行される. これは指数 y の二進法表現における 1 の個数に対応する. 答えは 2.

クイズ 5.1. 法 N で a の逆元が存在するための必要十分条件は

$$\gcd(a, N) = 1$$

である. 答えは 2.

クイズ 5.2. $5 \times 8 = 40 \equiv 1 \pmod{13}$ より, $5^{-1} \equiv 8 \pmod{13}$ である. したがって

$$7 \times 5^{-1} \equiv 7 \times 8 = 56 \equiv 4 \pmod{13}.$$

答えは 2.

クイズ 5.3. 逆元を持つのは, 21 と互いに素なものだけである.

$$\gcd(4, 21) = 1, \quad \gcd(6, 21) = 3, \quad \gcd(14, 21) = 7, \quad \gcd(15, 21) = 3.$$

よって, 逆元を持つのは 4 だけである. 答えは 1.

クイズ 7.1. 1 はフェルマーの小定理そのものであり正しい. 2 は正しくない. 合成数では一般には成り立たない. 3 は正しい. カーマイケル数では, N と互いに素なすべての a に対して

$$a^{N-1} \equiv 1 \pmod{N}$$

が成り立つ. 4 は正しくない. フェルマー検査は合成数を素数と誤判定することがある. 答えは 1, 3.

クイズ 7.2. 素数定理より, x 付近の整数が素数である確率はおおよそ $1/\ln x$ である. n ビット整数では $x \approx 2^n$ なので,

$$\frac{1}{\ln x} \approx \frac{1}{\ln 2^n} = \frac{1}{n \ln 2}.$$

したがって, 素数である確率はおおよそ $1/n$ である. 答えは 1.